Lattice · Feature deck

Native state machines in plain markdown.

A list of states, arrows to a state's name, palette-blind SVG. The flowchart's grammar, on the same graph library.

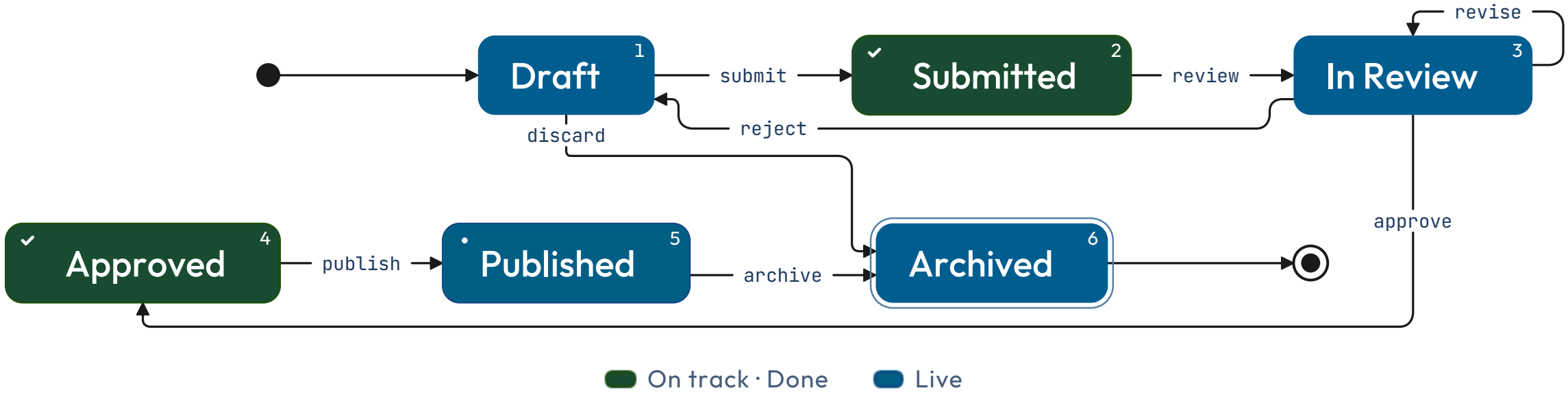
What this deck shows.

A finite-state machine, authored as a list. Each item is a state, named by its text; its badge shows its place in the list. A sub-item that starts with an arrow is a transition to a state named by its text: `-submit-> Submitted`, or the state's own name for a self-loop. A sub-list of states makes a composite state, and a `>` blockquote under a state is hidden detail. The browser measures the states and Trama lays the machine out: a chain wraps into rows, a machine that branches is ranked, and every line is an elbow that never crosses a state. `lr` and `tb` pin the direction, `inline` renders rows of chips, `curved` rounds the corners.

Submission lifecycle

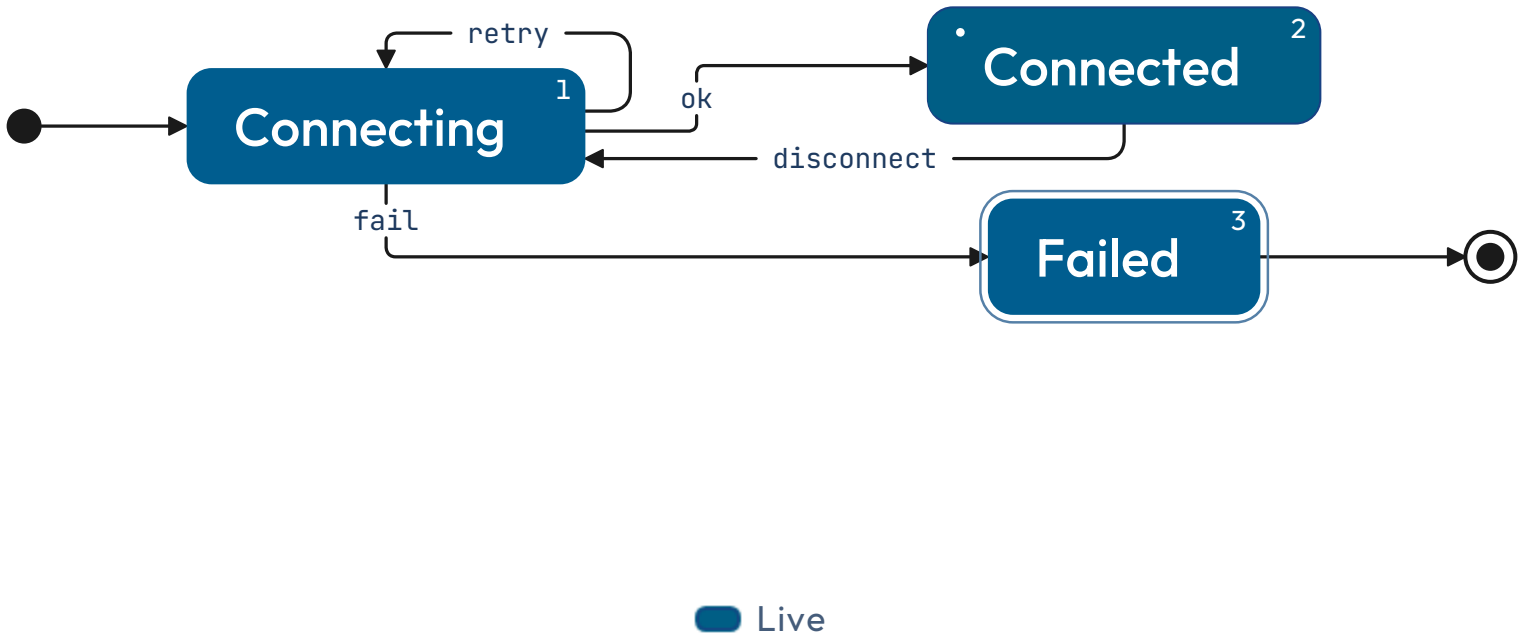
Document approval flow.

How a draft moves from author to archive.

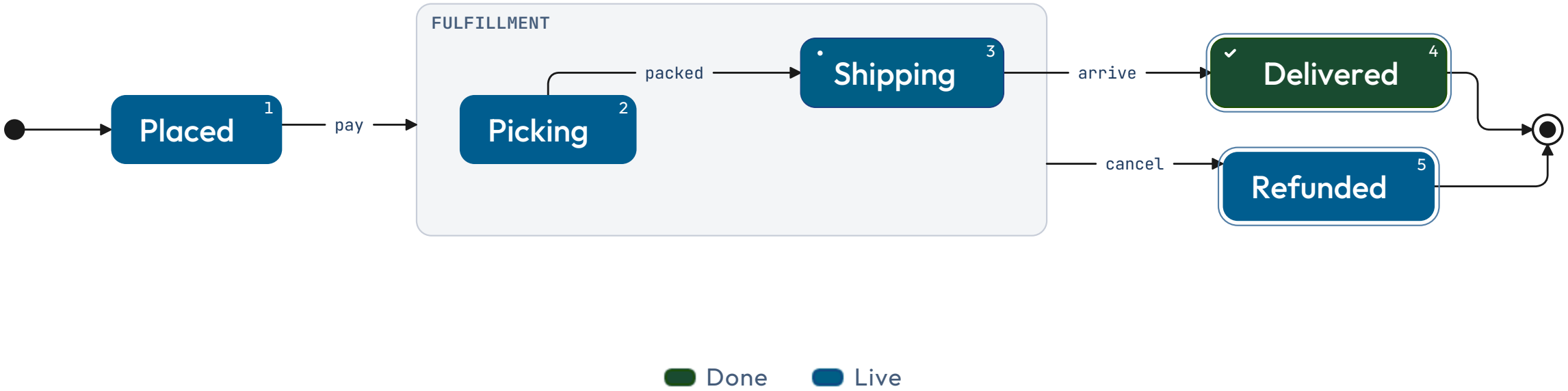


Rejected drafts return to the author; revisions stay in review.

Connection retry.

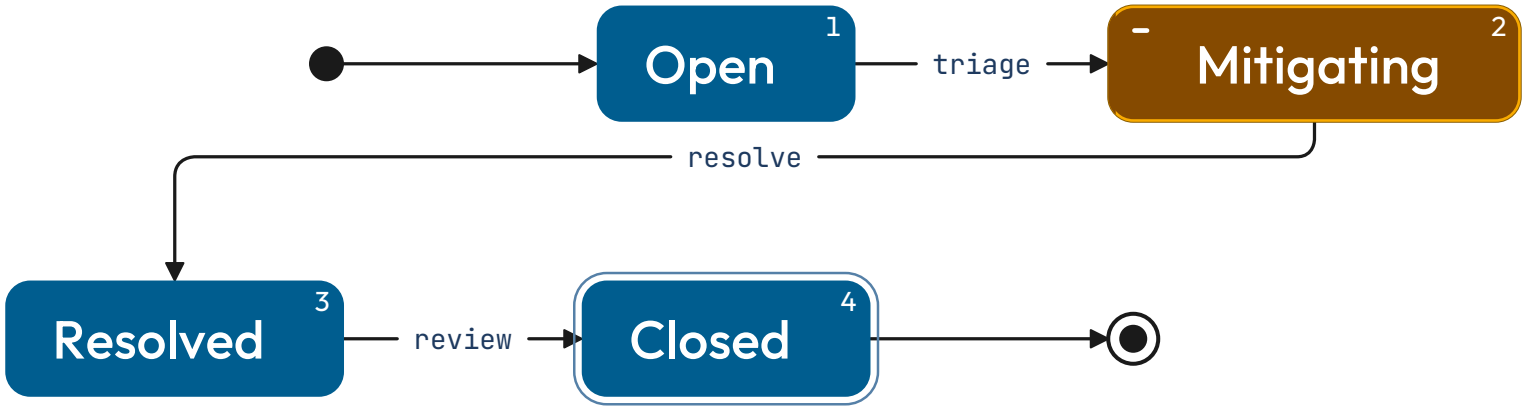


An order ships in stages.



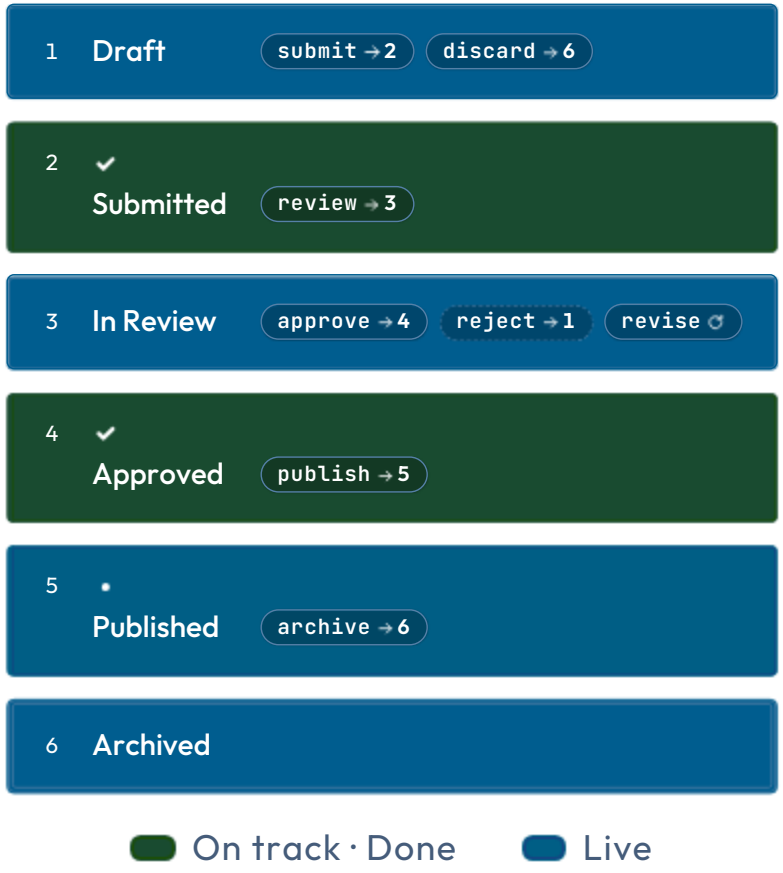
A transition from the composite leaves every state inside it.

Every incident ends in a review.

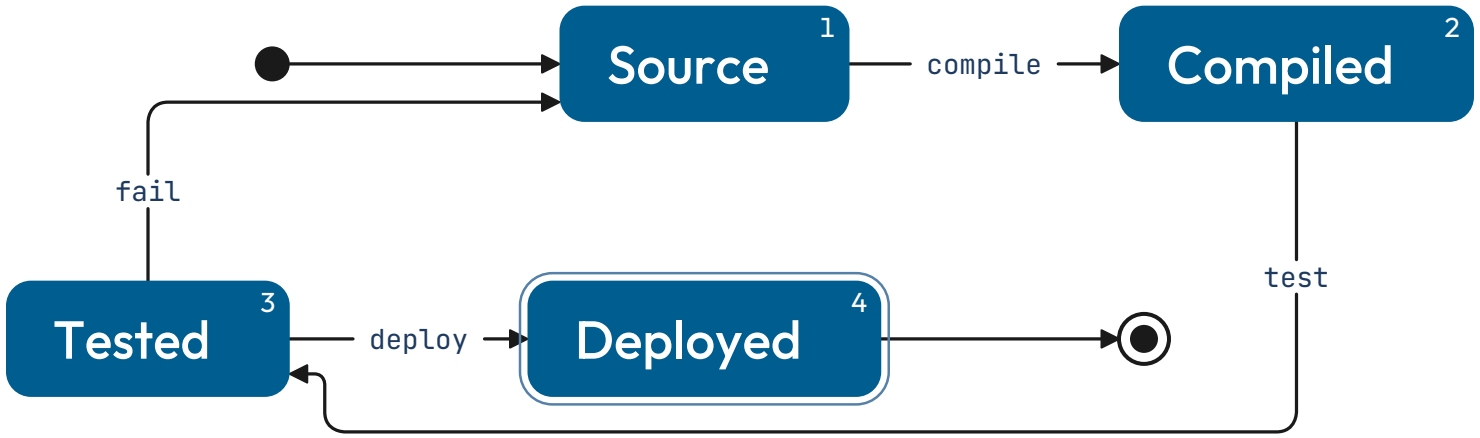


Hover a state in Present to read its detail.

Inline rendering.



Build pipeline.



lr — left-to-right (Mermaid direction LR)

Why a native state chart.

Mermaid's `stateDiagram-v2` works, but theming it cleanly requires a CSS override cascade with `!important` (see `docs/theming.md`). The native chart uses palette tokens directly — `var(--cat-1-fill)`, `var(--diagram-stroke)`, `var(--diagram-line)`, `var(--cat-on-fill)` — and inherits dark / light, every theme, automatically. No mmdc subprocess, no opaque SVG class names, no version coupling. Composite states are a sub-list; parallel regions, history states and guards are not in the grammar. When you need those, `<!-- _class: diagram -->` is the Mermaid escape hatch.

Lattice · State chart

The list is the reading order.